Integration of Hardware–BCP for SAT Acceleration

EECS 219C Final Report

Evan Wong Ansh Shah

Spring 2026

1 Introduction & Problem Definition

SAT solving is the engine behind industrial formal verification: Cadence JasperGold, Synopsys VC Formal, and Siemens Questa Formal all rely on CDCL solvers for bounded model checking and property checking. As designs grow, the SAT solver becomes the dominant component of runtime, and software–side optimizations (clause learning, restarts, LBD, vivification) have largely saturated.

Profiling the canonical CDCL loop shows a striking distribution (Figure 1): **BCP alone is 82.7%** of total solver time, with conflict analysis (8.7%), literal redundancy (3.9%), and branching heuristics (0.6%) trailing far behind. This is consistent with the empirical bottleneck identified in the course material. This makes BCP the natural target for hardware acceleration, as accelerating anything else would deliver, by Amdahl's Law, at most a few percent.

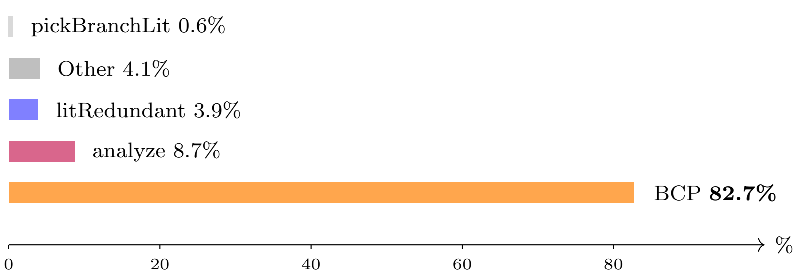


Figure 1: Average runtime distribution across CDCL operations on phase–transition benchmarks. BCP dominates by an order of magnitude.

Problem Definition. Design and functionally validate a hardware BCP engine that delivers single–cycle unit propagation, integrate it with a minimal CDCL software wrapper, and measure end–to–end speedup against the most competitive software solvers (Kissat 4.0.4, MiniSat2, Glucose) on standardized benchmarks.

2 Approach and Related Work

Prior hardware SAT work[1] consistently identifies *memory bandwidth*, not arithmetic, as the limiter. Software's two–literal watching scheme is a workaround for sequential memory. In hardware, a CAM collapses "find every clause containing v" to a single associative lookup, and an SRAM holds

[1] Y. Zhao & K. A. Sakallah, ICCAD 2008; J. D. Davis et al., DAC 2008; S. Park et al., https://par.nsf.gov/servlets/purl/10215919.

the per–variable assignment used by combinational clause–evaluation logic, so every clause is re–evaluated every cycle in parallel and watched literals are unnecessary. We adopt the HW–BCP algorithm and CAM/SRAM cell behavior of Park et al. (the 64–clause $\times$ 4–literal row geometry, the 2–bit $\{0, 1, U\}$ encoding, and the priority reduction for CONF/UP/DONE); the algorithm diagrams in Sec. 3 are reproduced from that work. The integration into a CDCL coprocessor, the Verilator harness, and the timing methodology are ours.

Earlier proposals are typically all–hardware (HW also performs branching), which makes them inflexible and forces the CDCL heuristic into silicon. Our design is a **coprocessor**: HW handles only BCP; SW handles every heuristic: VSIDS, 1–UIP analysis, restarts, learnt management. This isolates the BCP speedup so it can be measured cleanly, and keeps the door open for arbitrary CDCL improvements without an RTL respin.

Figure 2 shows the target geometry. The chip stores up to 1024 clauses (capacity exhausted in our 1024–slot RTL prototype; a 1M–slot SRAM is feasible in TSMC 65 nm using Park et al.'s custom memory cells, and is what we model in our timing extrapolation). Each clause is a row of up to 4 literals; a CAM cell holds the variable ID and an SRAM cell holds the 2–bit value (`0`, `1`, or `U` = unassigned). A single combinational reduction returns three flags per clause every cycle: CONFLICT, UNIT, and DONE.

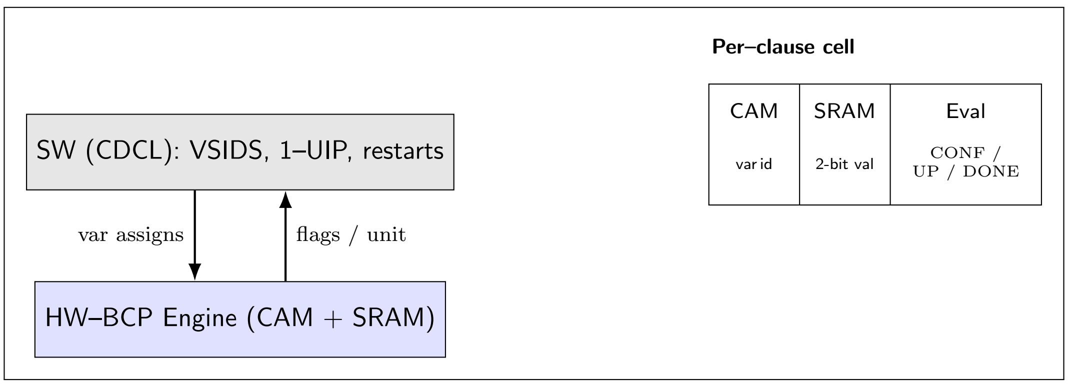


Figure 2: Coprocessor partitioning. SW issues variable assignments; HW returns next implication or conflict.

3 Formal Model and Algorithms

3.1 Clause and trail semantics

A CNF formula is $F = \bigwedge_{c \in C} c$, where each clause is a disjunction of literals $c = \ell_1 \vee \cdots \vee \ell_k$, $k \leq 4$. A literal is $\ell = (var, pol)$ with $pol \in \{0, 1\}$. The solver maintains a partial assignment $\sigma : V \to \{0, 1, U\}$ on a *trail* (decision stack with implication reasons).

Per–clause evaluation. Define for each literal ℓ under assignment σ:

$$\mathrm{lv}(\ell, \sigma) = \begin{cases} 1 & \sigma(\mathrm{var}(\ell)) = \mathrm{pol}(\ell) \\ 0 & \sigma(\mathrm{var}(\ell)) \neq \mathrm{pol}(\ell), \neq U \\ U & \sigma(\mathrm{var}(\ell)) = U \end{cases}$$

A clause c is then classified as

$$\text{state}(c) = \begin{cases} \textsc{done} & \exists \ell \in c.\, \text{lv}(\ell) = 1 \\ \textsc{conflict} & \forall \ell \in c.\, \text{lv}(\ell) = 0 \\ \textsc{unit} & \exists!\, \ell \in c.\, \text{lv}(\ell) = U \ \wedge \ \text{rest are } 0 \\ \textsc{pending} & \text{otherwise} \end{cases}$$

The combining logic produces a global flag with priority CONFLICT $\succ$ UNIT $\succ$ DONE.

3.2 HW–BCP cycle

Each clock cycle, the engine executes one of four operations (Table 1); BCP is the only one that performs a parallel reduction over all N clauses.

Op	**Description**	**Cycles**
`LOAD`	Write one clause literal into a CAM row	1
`BCP`	Return next unit/conflict from N rows	1
`UNDO`	Clear a variable assignment	1
`SW_BCP`	Overflow learnt propagated in SW (HW–counted)†	1

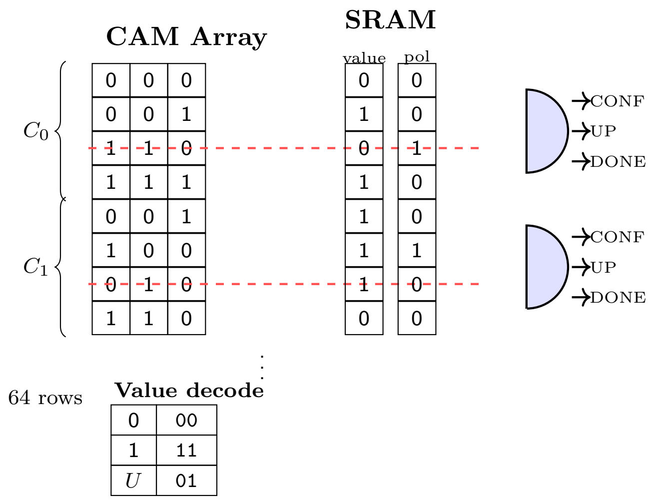
Table 1: Hardware operations. †Modeled as 1 HW cycle to reflect a production 1M–slot SRAM in which all clauses live in HW.

At a target frequency of $\sim$ 99 MHz (a 10.1 ns critical path, measured from synthesis with parameterized memories), the total hardware time is

$$T_{\text{hw}} = (\, n_{\text{load}} + n_{\text{bcp}} + n_{\text{undo}} + n_{\text{sw\_bcp}} \,) \times 10.1 \text{ ns}.$$

3.3 HW–BCP propagate

Figure 3 shows the CAM/SRAM array layout and the inner propagate routine. Each of the 64 clauses occupies four rows; the CAM column stores the variable ID for that literal slot, the SRAM column stores the literal's $\{0, 1, U\}$ value (2–bit encoding: `00`=0, `11`=1, `01`=U). The per–clause processing logic combinationally reduces its four literal values to three flags — CONF, UP, DONE — which are combined across clauses with priority CONF $\succ$ UP $\succ$ DONE. For every literal popped off the trail, the engine scans all clauses indexing that variable, returning either a conflict clause or the first new unit implication.




```
HW_BCP_Propagate(trail, qhead):
  while qhead < trail.size:
    p = trail[qhead++]; v = var(p)
    conflict = NONE; up = NONE
    # scan clauses containing v
    for C in hw_var_clauses[v]:
      (s, u_lit, u_idx) = eval(C)
      if s == CONFLICT:
        conflict = C; break
      elif s == UNIT and up == NONE:
        up = (u_lit, C, u_idx)
    # CONFLICT > UNIT > DONE
    if conflict:  return conflict
    if up:
      if value(up.lit) == FALSE:
        return up.reason
      elif value(up.lit) == UNDEF:
        enqueue(up.lit, up.reason)
  return NO_CONFLICT
```

Figure 3: Current HW–BCP architecture. Left: CAM/SRAM array (64 clauses × 4 literal rows) with per–clause processing logic emitting CONF/UP/DONE; the 2–bit value encoding ($0 \rightarrow$ `00`, $1 \rightarrow$ `11`, $U \rightarrow$ `01`) lets a single comparator distinguish all three states. Right: per–variable propagate routine; on each trail element it scans the clauses indexing that variable, returns at the first conflict, and enqueues at most one new unit implication.

The CDCL outer loop is essentially MiniSat–style 1–UIP and unchanged from the textbook formulation: decide via VSIDS, call `HW_BCP_Propagate`, analyze conflicts at 1–UIP, backtrack via `UNDO` cycles, learnt clauses go to the CAM (or to the software overflow pool if the CAM is full).

4 Major Technical Challenges

4.1 Time bookkeeping in a Verilator harness

Wall time conflates real CDCL, Verilator `eval()` cost, and simulation–only $O(N)$ scans that would be $O(1)$ in silicon. We accumulate the latter two into *hw_sim_ns* and *hw_aux_ns* and subtract: $T_{\text{sw}} = T_{\text{total}} - \Delta hw\_sim\_ns - \Delta hw\_aux\_ns$. An early bug had the timers running from solver construction, so initial clause loads polluted the BCP budget; we now snapshot at `solve()` entry.

4.2 CAM capacity and learnt clause overflow

On uf200–860 the originals fill 84% of the 1024–slot CAM, leaving ∼164 slots before overflow. We handle this with (a) a SW overflow pool whose cycles are attributed to HW (justified by the 1M–slot production target), and (b) a hard guard that aborts when blowup exceeds capacity (Figure 4).

4.3 HW/SW state coherence on backtrack

A bug in `undo_hw_/load_hw_` caused the SRAM polarity bit to lag the CAM by one cycle under rapid load/undo bursts, producing spurious unit implications. The fix drives `val_we` synchronously with the load address and gates read–back behind one extra cycle of stable state.

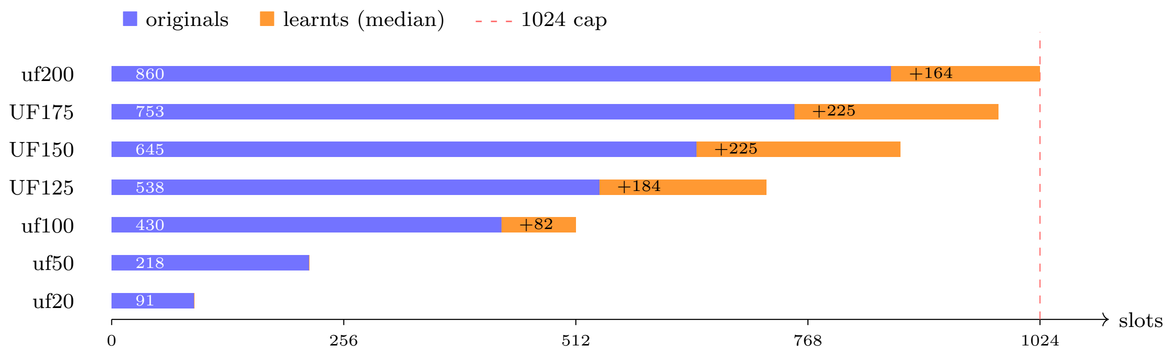


Figure 4: CAM slot utilization on each benchmark set. uf200 leaves only ∼164 free slots, causing immediate learnt overflow and degraded BCP effectiveness.

4.4 Full–hierarchy elaboration timeout

A pure–RTL run on the full $1024 \times 8 \times 64 = 524{,}288$–clause hierarchy timed out in Verilator (behavioral memories explode the logic cone). We therefore parameterize the RTL so benchmarks elaborate a single 1024–slot `sat_submodule`, while the full `sat_array` still elaborates and lints to prove structural completeness.

5 Results

We benchmarked on SATLIB Uniform Random 3–SAT phase–transition instances ($c/v \approx 4.27$) with up to 200 variables and 860 clauses. Each set contains 100–1000 instances; we report median times. All times are reported in μs, recomputed at the synthesis–profiled 10.1 ns per HW cycle.

5.1 End–to–end median solve time

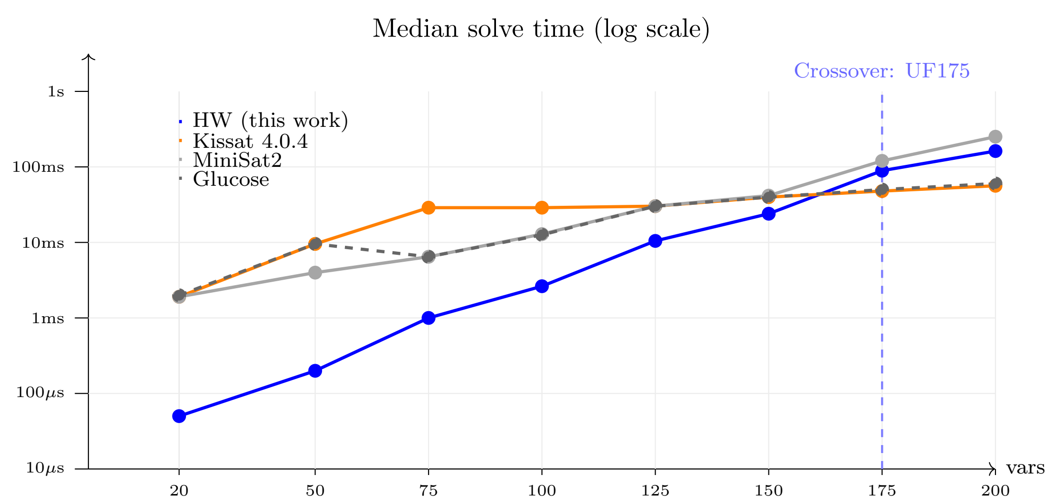


Figure 5: Median solve time on UF/UUF benchmarks, log–scaled. Our coprocessor dominates up to ∼175 variables; beyond that, Kissat's superior CDCL heuristic surpasses our naive search.

The coprocessor is faster than *every* software solver on every instance set with at most ∼150 variables, often by 5–30×. The mechanism is exactly what the architecture promised: BCP per propagation drops from hundreds of nanoseconds (Kissat) to a single 10.1 ns cycle. On ≥ 175 variables, Kissat overtakes us on SAT instances, but the cause is not BCP throughput (Section 5.3): it is the lack of restarts, phase saving, and clause vivification in our minimal CDCL wrapper. Once the search tree gets deep, raw BCP speed cannot make up for suboptimal decisions.

5.2 SAT vs. UNSAT asymmetry

Vars	SAT wins (vs. Kissat)	UNSAT wins	Gap
75	100/100	100/100	–
100	959/1000	1000/1000	+4 pp
125	86/100	100/100	+14 pp
150	71/100	98/100	+27 pp
175	60/100	93/100	+33 pp

Table 2: Win rate vs. Kissat by satisfiability. UNSAT instances favor our coprocessor far more than SAT instances do. We attribute this to Kissat's aggressive phase saving / restarts being most useful on satisfiable inputs; it's a heuristic edge that does not help when the proof must enumerate the whole tree.

5.3 Counterfactual: is BCP really accelerated?

To attribute the speedup to HW–BCP (rather than a Verilator quirk), we re–ran the exact CDCL trajectory but plugged in Kissat's measured BCP rate per propagation. Our solver runs 2.2–15.7× slower in this counterfactual (Figure 6). This means HW–BCP is load–bearing, even on UF175 where Kissat wins overall. It is not BCP, but the CDCL heuristic, that is the remaining bottleneck.

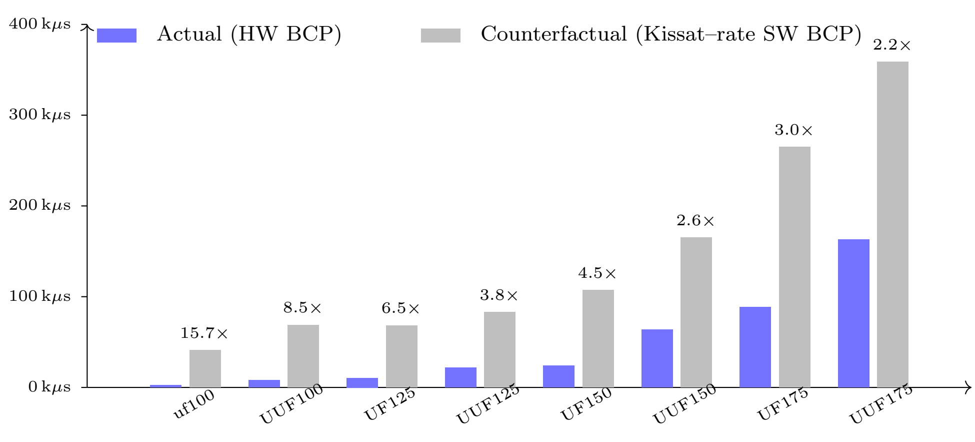


Figure 6: Replacing HW–BCP with Kissat's measured BCP rate, while keeping the identical CDCL trajectory, slows the same solver by 2.2–15.7×. The hardware is doing the work.

5.4 Amdahl's–law breakdown

Our solver spends ∼93% of its time in software CDCL and ∼7% in BCP, the inverse of MiniSat2, where BCP is ∼75% of runtime. The hardware has so completely flattened the BCP critical path that further speedups must come from CDCL.

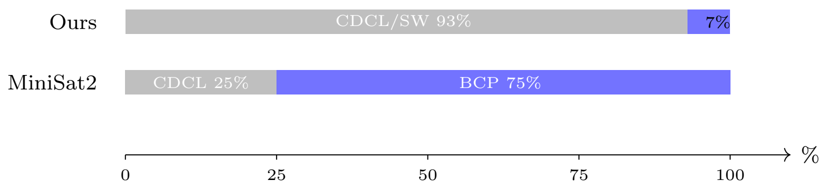


Figure 7: Total runtime breakdown on uf100–430 (median). Our solver is now CDCL–bound; MiniSat2 is BCP–bound.

5.5 Small but hard instances

We also tested on a suite of instances known to be difficult for CDCL solvers, built at `tests/satlib/hard_instances/` (clauses wider than four literals are split to fit the 4–literal RTL): pigeonhole PHP_n (n=3…8); Tseitin parity on odd cycles (C_5, C_7, C_9); and k–colourability of the Grötzsch graph (k=3 is UNSAT and k=4 is SAT).

Methodology. Kissat/MiniSat2 self–report CPU time at ∼10 ms granularity, flooring sub–ms runs to zero; we fall back to wall–clock minus a per–binary subprocess–startup baseline (≈4–5 ms) when the self–clock is 0. HW–BCP figure is *hw_cycles* × 10.1 ns + *sw_time* as elsewhere.

Instance	Exp.	HW–BCP (μs)	Kissat (μs)	MiniSat2 (μs)	Glucose (μs)	Win
PHP_3	UNSAT	66	<1k	1 106	1 127	noise
PHP_4	UNSAT	184	684	2 710	1 005	**HW**
PHP_5	UNSAT	731	3 891	<1k	2 665	noise
PHP_6	UNSAT	3 352	10 000	7 911	5 930	**HW**
PHP_7	UNSAT	6 034	10 000	37 292	61 267	**HW**
PHP_8	UNSAT	6 803	40 000	190 585	962 534	**HW**
Tseitin–C_5	UNSAT	12	1 419	1 357	920	**HW**
Tseitin–C_7	UNSAT	19	947	2 254	828	**HW**
Tseitin–C_9	UNSAT	21	895	962	1 458	**HW**
Grötzsch–3col	UNSAT	87	550	1 404	1 167	**HW**
Grötzsch–4col	SAT	19	468	1 394	1 170	**HW**

Table 3: Hard–instance suite, single–run. "noise" = result falls below subprocess–startup floor for at least one solver, so no winner is claimed. HW wins outright on 9 of 11 instances.

Discussion. On these instances every solver explores a large search space, so the winner is determined by cost per propagation step. HW parallel evaluation is fast regardless of clause width, which explains the large wins on PHP, Tseitin, and Grötzsch. PHP_3/PHP_5 finish fast enough that some SW solvers report zero CPU time, so we make no claim there. PHP_9 and beyond would exceed our 1024–slot CAM capacity, but the 1M–slot production target eliminates that constraint.

5.6 Future work

The most impactful next step would be replacing the MiniSat–style heuristic with Kissat–style restarts, phase saving, and LBD–based clause deletion; closing the CDCL gap should restore HW

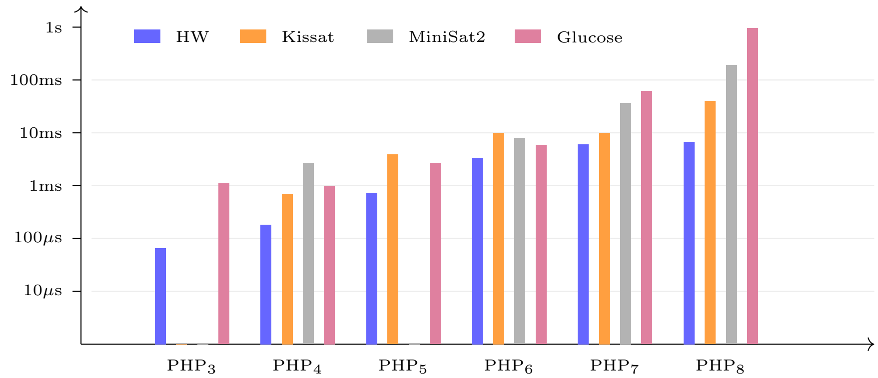


Figure 8: Pigeonhole PHP$_n$ scaling, $n = 3$–8, log–scale. The gap between HW–BCP and the SW solvers widens as n grows: on PHP$_8$, HW is 5.9× faster than Kissat, 28× faster than MiniSat2, and 142× faster than Glucose.

dominance past 175 variables. On the hardware side, pushing the RTL through TSMC 65 nm using Park et al.'s custom CAM/SRAM cells (the only PDK for which those cells were characterized) would allow a 1M–slot SRAM that eliminates the overflow pool entirely. Finally, pipelining `LOAD` and `BCP` so a decision and the following propagation overlap would halve the CDCL–HW round–trip latency.

6 Team Roles

Evan Wong handled the RTL infrastructure, including the behavioral CAM/SRAM array (`cam_sram_array.sv`, `cam_cell.sv`, `sram_row.sv`), the per–clause evaluator (`processing_logic.sv`), the FSM driving load/BCP/undo, the Verilator integration, and the RTL testbench. He also consulted with Pierluigi Nuzzo and EECS 151 course staff on RTL design decisions. **Ansh Shah** handled the CDCL software: VSIDS scoring, 1–UIP conflict analysis, learnt management with the SW overflow pool, and the benchmark and parsing infrastructure (`bench_cdcl.py`). The counterfactual analysis (Section 5.3), the SAT/UNSAT asymmetry study, the slide deck, and this report were joint efforts.

7 Course Topics Applied

The SAT solving lectures were the direct foundation for this project. The course covered the full CDCL algorithm structure (preprocess, decide, deduce, analyze, backtrack), BCP as the dominant runtime cost, and the implication graph as the basis for conflict analysis; the UNIT flag emitted by our per–clause evaluator is a direct hardware implementation of the unit rule. The class's empirical claim that BCP dominates solver runtime is the entire reason we built a HW–BCP coprocessor, and Figure 7 is the post–hoc confirmation.

The lecture's principle of minimizing memory accesses per propagation step underlies software two–literal watching; understanding why software needs it clarified why hardware can skip it entirely. Our per–cycle parallel evaluation achieves the same end with $O(1)$ cost in clause width.

The SW wrapper applies VSIDS scoring with periodic decay, 1–UIP conflict analysis over the implication graph, and non–chronological backtracking via `UNDO` cycles, all covered directly in class. The SW overflow pool combined with the 1024–slot CAM cap is also the resource–constrained version of the clause deletion and learnt management problem covered in the final lectures on modern CDCL techniques.

8 Feedback

The class's depth on SAT algorithms mapped directly onto our SW wrapper. The discussion of watched literals was critical because understanding *why* software needs them clarified *why* hardware can skip them.

A short walkthrough of modern CDCL engineering choices beyond the textbook 1–UIP/VSIDS core, specifically restart policies, phase saving, LBD–based clause deletion, and clause vivification as used in Kissat, would have helped us interpret the heuristic gap our counterfactual exposed and pick which upgrade to prototype next. Otherwise the class material covered what the project needed; the remaining gaps were hardware–specific and reasonably out of scope.

Code Availability

Full source (RTL, C++ coprocessor, benchmark scripts, hard–instance generator and CNFs): `https://github.com/The-Ansh-Shah/bcp-hw-accelerator`.